Оглавление

Введение 2
1 ГЛАВА 1 ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ КЛАССИФИКАЦИИ ИЗОБРАЖЕНИЙ И ТЕКСТОВЫХ ДАННЫХ 6
1.1 Понятие и задачи классификации данных 6
1.2 Методы и алгоритмы классификации текстовых данных 7
1.3 Методы классификации изображений 9
1.4 Современные технологии машинного обучения для анализа данных 10
1.5 Использование языка Python для разработки систем машинного обучения 11
1.6 Применение low-code платформ для разработки пользовательских интерфейсов 12
1.7 Анализ существующих решений в области классификации данных 13
1.8 Выводы по первой главе 14
2 ГЛАВА 2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ КЛАССИФИКАЦИИ ИЗОБРАЖЕНИЙ И ТЕКСТОВЫХ ДАННЫХ 15
2.1 Постановка задачи разработки системы 15
2.2 Формирование требований к системе 16
2.3 Общая архитектура программной системы 17
2.4 Проектирование структуры программного обеспечения 19
2.5 Проектирование алгоритма работы системы 20
2.6 Проектирование взаимодействия пользовательского интерфейса и серверной части 22
2.7 Выбор технологий и инструментальных средств разработки 22
2.8 Выводы по второй главе 23

Введение

В современных условиях стремительного развития информационных технологий происходит постоянный рост объёмов цифровой информации. Значительную часть таких данных составляют текстовые документы и графические материалы, включая фотографии, изображения и различные визуальные объекты. В связи с этим возникает необходимость разработки программных средств, способных автоматически анализировать и классифицировать большие массивы информации. Одним из наиболее перспективных направлений решения данной задачи является использование методов машинного обучения и интеллектуального анализа данных.

Классификация данных представляет собой процесс автоматического определения принадлежности объекта к определённому классу на основании его характеристик. Данная задача широко применяется в различных областях, включая системы поиска информации, обработку мультимедийных данных, анализ социальных сетей, медицинскую диагностику, системы рекомендаций и информационную безопасность. Особую актуальность приобретают системы, способные одновременно работать с различными типами данных, например текстовыми и графическими.

Классификация изображений является одной из ключевых задач компьютерного зрения. Современные методы анализа изображений позволяют автоматически распознавать объекты, определять категории изображений и выполнять их автоматическую сортировку. В последние годы значительное развитие получили методы глубокого обучения, в частности свёрточные нейронные сети, которые обеспечивают высокую точность распознавания изображений.

Классификация текстовых данных относится к области обработки естественного языка и используется для автоматического анализа содержимого текстов. Такие методы применяются для определения тематики документов,

фильтрации спама, анализа тональности сообщений и категоризации информации. Для решения подобных задач используются различные алгоритмы машинного обучения, включая вероятностные модели, методы линейной классификации и нейронные сети.

Развитие технологий машинного обучения и наличие специализированных библиотек программирования позволяют создавать интеллектуальные системы анализа данных на основе современных алгоритмов. Одним из наиболее распространённых инструментов разработки таких систем является язык программирования Python. Популярность Python обусловлена простотой синтаксиса, широкими возможностями работы с данными и наличием большого количества библиотек для машинного обучения и компьютерного зрения.

Помимо разработки алгоритмов обработки данных важную роль играет создание удобного пользовательского интерфейса, обеспечивающего взаимодействие пользователей с системой. В последние годы активно развиваются low-code платформы, позволяющие создавать интерфейсы программных систем с минимальным объёмом программирования. Использование подобных платформ позволяет значительно сократить время разработки программного обеспечения и повысить доступность систем анализа данных для конечных пользователей.

Таким образом, разработка программной системы, объединяющей методы классификации изображений и текстовых данных с использованием современных инструментов машинного обучения и удобного пользовательского интерфейса, является актуальной задачей.

Целью выпускной квалификационной работы является разработка программной системы классификации изображений и текстовых данных с использованием языка программирования Python и пользовательского интерфейса, реализованного на low-code платформе.

Для достижения поставленной цели необходимо решить следующие задачи:

1. провести анализ методов классификации текстовых и графических данных;
2. изучить современные технологии машинного обучения, применяемые для анализа данных;
3. рассмотреть инструменты разработки систем машинного обучения на языке Python;
4. разработать архитектуру программной системы классификации данных;
5. реализовать программную систему классификации изображений и текстовых данных;
6. выполнить тестирование разработанной системы и оценить результаты её работы.

Объектом исследования являются методы автоматической классификации данных.

Предметом исследования являются алгоритмы и программные средства классификации изображений и текстовой информации.

Методы исследования, используемые в работе, включают методы машинного обучения, методы обработки текстовых данных, методы компьютерного зрения и методы проектирования программных систем.

Практическая значимость работы заключается в разработке программной системы, которая может быть использована для автоматизированной обработки и классификации мультимедийных данных.

Структура выпускной квалификационной работы включает введение, четыре главы основной части, заключение, список использованных источников и приложение. В первой главе проводится анализ предметной области классификации изображений и текстовых данных. Во второй главе рассматриваются вопросы проектирования программной системы, включая архитектуру, варианты использования и структуру базы данных. В третьей главе

описывается практическая реализация программного продукта и интерфейса. В четвертой главе приводятся результаты тестирования и оценки качества разработанной системы.

1 ГЛАВА 1 ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ КЛАССИФИКАЦИИ ИЗОБРАЖЕНИЙ И ТЕКСТОВЫХ ДАННЫХ

1.1 Понятие и задачи классификации данных

В условиях стремительного роста объёмов цифровой информации важной задачей становится автоматизированная обработка и анализ данных. Значительная часть информации, циркулирующей в современных информационных системах, представлена в виде текстовых документов, изображений, мультимедийных файлов и других видов неструктурированных данных. Для эффективной работы с такими данными применяются методы интеллектуального анализа, одним из наиболее распространённых из которых является классификация.

Классификация представляет собой процесс отнесения объектов к определённым категориям или классам на основе анализа их характеристик. В рамках задач машинного обучения классификация относится к типу задач обучения с учителем, при котором модель обучается на заранее размеченных данных, содержащих примеры объектов и соответствующие им классы.

Основной целью классификации является построение модели, способной на основе имеющихся признаков объекта определить его принадлежность к одному из заранее заданных классов. Такая модель может использоваться для анализа новых данных, которые не участвовали в процессе обучения.

Задачи классификации широко применяются в различных областях, включая обработку текстовой информации, анализ изображений, медицинскую диагностику, финансовый анализ, системы рекомендаций и системы информационной безопасности. В частности, автоматическая классификация изображений позволяет распознавать объекты на фотографиях, а классификация

текстов применяется для определения тематики документов, анализа пользовательских отзывов и фильтрации нежелательного контента.

Современные системы анализа данных часто работают одновременно с несколькими типами информации, что требует разработки универсальных методов обработки данных различной природы.

1.2 Методы и алгоритмы классификации текстовых данных

Классификация текстовых данных является одной из ключевых задач обработки естественного языка. Основной особенностью текстовой информации является её неструктурированный характер, что требует выполнения предварительной обработки данных перед применением алгоритмов машинного обучения.

Первым этапом обработки текстов обычно является очистка данных, включающая удаление служебных символов, приведение текста к единому регистру, удаление стоп-слов и проведение лемматизации или стемминга. После этого текст преобразуется в числовое представление, которое может быть использовано алгоритмами машинного обучения.

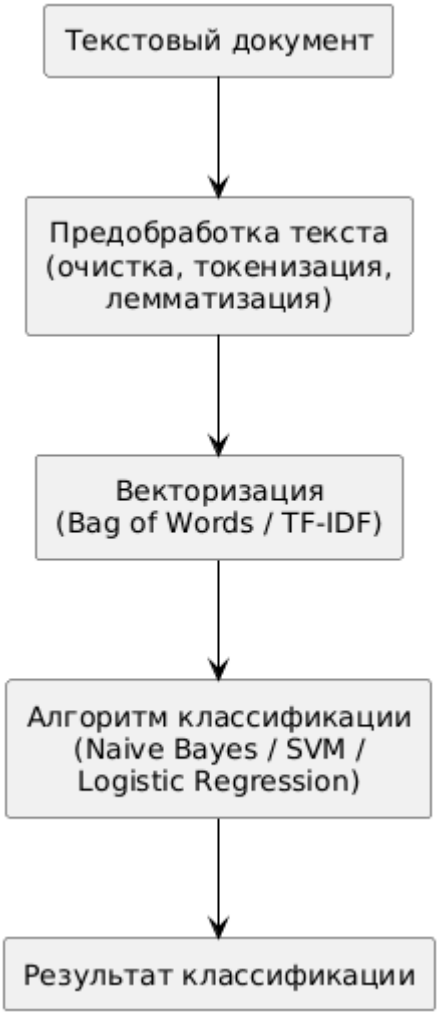

Рисунок 1 Общая схема классификации текстовых данных

Одним из наиболее распространённых методов представления текста является модель «мешка слов» (Bag of Words). В рамках данной модели текст представляется в виде набора слов, а каждому слову соответствует частота его появления в документе.

Более информативным методом является использование меры TF-IDF, которая учитывает как частоту появления слова в документе, так и его распространённость в коллекции текстов. Это позволяет снизить влияние часто встречающихся слов, не несущих значительной смысловой нагрузки.

В качестве алгоритмов классификации текстов широко применяются различные методы машинного обучения, среди которых можно выделить наивный байесовский классификатор, метод опорных векторов, логистическую

регрессию и алгоритмы ансамблевого обучения. В последние годы значительное распространение получили нейронные сети, позволяющие учитывать контекст слов и их семантические взаимосвязи.

Современные методы обработки текстов всё чаще используют векторные представления слов, такие как word embeddings, которые позволяют представлять слова в виде многомерных числовых векторов. Такие представления учитывают контекстное использование слов и позволяют повысить точность классификации.

1.3 Методы классификации изображений

Классификация изображений является одной из основных задач компьютерного зрения. Она заключается в автоматическом определении категории, к которой относится изображение на основании анализа его визуальных характеристик.

Рисунок 2 Архитектура сверточной нейронной сети для классификации изображений

Традиционные методы обработки изображений основывались на использовании заранее заданных признаков, которые извлекались из изображений вручную. К таким признакам относятся цветовые характеристики, текстурные признаки, формы объектов и различные геометрические параметры.

Однако разработка эффективных признаков требует значительных усилий и часто зависит от особенностей конкретной задачи. В связи с этим традиционные методы имеют ограниченную универсальность и точность.

Значительный прогресс в области анализа изображений был достигнут благодаря развитию методов глубокого обучения. В частности, широкое распространение получили свёрточные нейронные сети, которые способны

автоматически извлекать признаки изображений на различных уровнях абстракции.

Свёрточные нейронные сети состоят из нескольких типов слоев, включая свёрточные слои, слои подвыборки и полносвязные слои. Свёрточные слои выполняют операцию свёртки, позволяющую выделять локальные признаки изображения, такие как границы, текстуры и формы объектов. По мере увеличения глубины сети признаки становятся более сложными и абстрактными.

Использование глубоких нейронных сетей позволило существенно повысить точность классификации изображений и расширить область применения систем компьютерного зрения.

1.4 Современные технологии машинного обучения для анализа данных

Машинное обучение является одним из ключевых направлений современной информатики, направленным на разработку алгоритмов, способных автоматически выявлять закономерности в данных и принимать решения на их основе.

Алгоритмы машинного обучения условно разделяются на несколько основных категорий. К первой категории относятся методы обучения с учителем, при которых модель обучается на размеченных данных. К таким методам относятся алгоритмы классификации и регрессии.

Ко второй категории относятся методы обучения без учителя, используемые для выявления скрытых структур в данных. Наиболее распространёнными задачами данного типа являются кластеризация и снижение размерности данных.

К третьей категории относятся методы обучения с подкреплением, при которых система обучается на основе взаимодействия с окружающей средой и получения вознаграждения за правильные действия.

Для решения задач классификации текстов и изображений наиболее часто используются методы обучения с учителем, поскольку они позволяют строить модели, способные точно определять принадлежность объектов к заданным классам.

Современные системы анализа данных активно используют методы глубокого обучения, основанные на применении многослойных нейронных сетей. Такие методы позволяют автоматически извлекать сложные признаки из исходных данных и обеспечивают высокую точность анализа.

1.5 Использование языка Python для разработки систем машинного обучения

Язык программирования Python является одним из наиболее распространённых инструментов разработки систем анализа данных и машинного обучения. Его популярность обусловлена простотой синтаксиса, гибкостью и наличием большого количества специализированных библиотек.

Python активно используется в научных исследованиях, разработке программных систем и анализе данных. Одним из ключевых преимуществ Python является развитая экосистема библиотек, предназначенных для решения различных задач обработки данных.

К числу наиболее популярных библиотек для анализа данных относятся NumPy и Pandas, которые обеспечивают эффективную работу с массивами данных и таблицами. Для реализации алгоритмов машинного обучения широко используется библиотека Scikit-learn, содержащая большое количество готовых моделей и инструментов для их обучения.

Для разработки нейронных сетей и реализации методов глубокого обучения применяются фреймворки TensorFlow и PyTorch. Эти инструменты позволяют создавать сложные модели машинного обучения и эффективно обучать их на больших наборах данных.

Для обработки изображений часто используется библиотека OpenCV, предоставляющая широкий набор инструментов компьютерного зрения.

Использование Python позволяет реализовать полный цикл разработки системы машинного обучения — от подготовки данных и обучения моделей до их интеграции в программные приложения.

1.6 Применение low-code платформ для разработки пользовательских интерфейсов

Современные программные системы требуют удобного и интуитивно понятного пользовательского интерфейса, обеспечивающего эффективное взаимодействие пользователя с системой. Разработка интерфейсов традиционными методами может требовать значительных временных и трудовых затрат.

Одним из перспективных подходов к созданию пользовательских интерфейсов является использование low-code платформ. Такие платформы предоставляют визуальные средства разработки приложений и позволяют создавать программные решения с минимальным объёмом программного кода.

Low-code платформы позволяют разрабатывать веб-приложения, информационные панели и интерфейсы взаимодействия пользователей с информационными системами. Основным преимуществом таких решений является возможность быстрого создания прототипов и сокращение времени разработки программного обеспечения.

Использование low-code платформ в сочетании с серверной частью, реализованной на языке Python, позволяет эффективно разделить задачи обработки данных и разработки пользовательского интерфейса. При этом серверная часть отвечает за выполнение алгоритмов машинного обучения и обработку данных, а пользовательский интерфейс обеспечивает удобное взаимодействие пользователей с системой.

1.7 Анализ существующих решений в области классификации данных

В настоящее время существует большое количество программных систем и библиотек, предназначенных для решения задач классификации данных. Многие из них реализованы в виде специализированных фреймворков машинного обучения или облачных сервисов анализа данных.

Рисунок 3 Обобщенная схема современных систем машинного обучения

К числу наиболее известных инструментов относятся библиотеки машинного обучения Python, такие как Scikit-learn, TensorFlow и PyTorch. Они предоставляют широкий набор алгоритмов для анализа данных и позволяют создавать модели классификации различной сложности.

Кроме того, существуют облачные сервисы машинного обучения, предоставляющие готовые инструменты анализа данных и обработки изображений. Такие сервисы позволяют использовать мощные вычислительные ресурсы и готовые модели машинного обучения без необходимости разработки сложных алгоритмов.

Несмотря на наличие большого количества инструментов, разработка специализированных систем классификации данных остаётся актуальной задачей, поскольку многие прикладные задачи требуют создания индивидуальных решений, адаптированных к конкретным условиям использования.

1.8 Выводы по первой главе

В первой главе был проведён обзор предметной области классификации данных. Рассмотрены основные понятия и задачи классификации, а также методы обработки текстовых и графических данных.

Проанализированы современные алгоритмы машинного обучения, применяемые для решения задач классификации, включая методы обработки текстовой информации и методы компьютерного зрения. Рассмотрены возможности языка программирования Python как основного инструмента разработки систем машинного обучения.

Также были рассмотрены особенности использования low-code платформ для разработки пользовательских интерфейсов программных систем.

Проведённый анализ позволяет сделать вывод о целесообразности разработки программной системы классификации изображений и текстовых данных с использованием современных методов машинного обучения и технологий разработки программного обеспечения. Полученные результаты анализа будут использованы при проектировании программной системы в следующей главе работы.

2 ГЛАВА 2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ КЛАССИФИКАЦИИ ИЗОБРАЖЕНИЙ И ТЕКСТОВЫХ ДАННЫХ

2.1 Постановка задачи разработки системы

В условиях постоянного увеличения объёмов цифровой информации возрастает необходимость создания программных систем, обеспечивающих автоматизированную обработку и анализ данных. Одной из наиболее актуальных задач является разработка программных средств, способных выполнять классификацию различных типов информации, включая текстовые данные и изображения.

Целью разработки является создание программной системы, позволяющей автоматически определять принадлежность текстовых данных и изображений к определённым категориям на основе методов машинного обучения.

Разрабатываемая система должна обеспечивать выполнение следующих функций:

- загрузку текстовых данных и изображений пользователем;
- предварительную обработку входных данных;
- анализ данных с использованием моделей машинного обучения;
- определение класса объекта;
- отображение результатов классификации пользователю.

Система должна обеспечивать возможность взаимодействия пользователя с программным продуктом через удобный пользовательский интерфейс, реализованный на основе low-code платформы.

2.2 Формирование требований к системе

При проектировании программной системы необходимо определить основные функциональные и нефункциональные требования, которым должна соответствовать разрабатываемая система.

К основным функциональным требованиям относятся:

- возможность загрузки пользователем текстовых данных для их анализа;
- возможность загрузки изображений для выполнения классификации;
- автоматическая предварительная обработка входных данных;
- выполнение классификации с использованием обученных моделей машинного обучения;
- отображение результатов анализа пользователю.

Нефункциональные требования включают требования к производительности, удобству использования и масштабируемости системы.

Система должна обеспечивать достаточно высокую скорость обработки данных, что позволит использовать её для анализа больших объёмов информации. Интерфейс пользователя должен быть простым и интуитивно понятным, что обеспечит удобство работы пользователей, не обладающих специальными знаниями в области машинного обучения.

Также система должна иметь модульную архитектуру, позволяющую при необходимости расширять её функциональность и интегрировать дополнительные алгоритмы анализа данных.

2.3 Диаграмма вариантов использования системы

Для уточнения функций будущего программного продукта была построена диаграмма вариантов использования. На ней отражены основные действия пользователя: классификация текста, классификация изображения, просмотр истории обращений и просмотр метрик обученных моделей.

Наличие диаграммы вариантов использования позволило сформировать минимально достаточный набор пользовательских сценариев, который затем был напрямую перенесен в структуру интерфейса Gradio и в набор автотестов.

Рисунок 2.1 - Диаграмма вариантов использования системы классификации данных

2.4 Общая архитектура программной системы

Разрабатываемая система классификации данных будет иметь модульную архитектуру, включающую несколько взаимосвязанных компонентов.

Основными компонентами системы являются:

- пользовательский интерфейс;
- серверная часть приложения;
- модуль предварительной обработки данных;
- модуль машинного обучения;
- модуль хранения данных.

Пользовательский интерфейс обеспечивает взаимодействие пользователя с системой. Через интерфейс пользователь может загрузить текст или изображение, запустить процесс анализа и получить результат классификации.

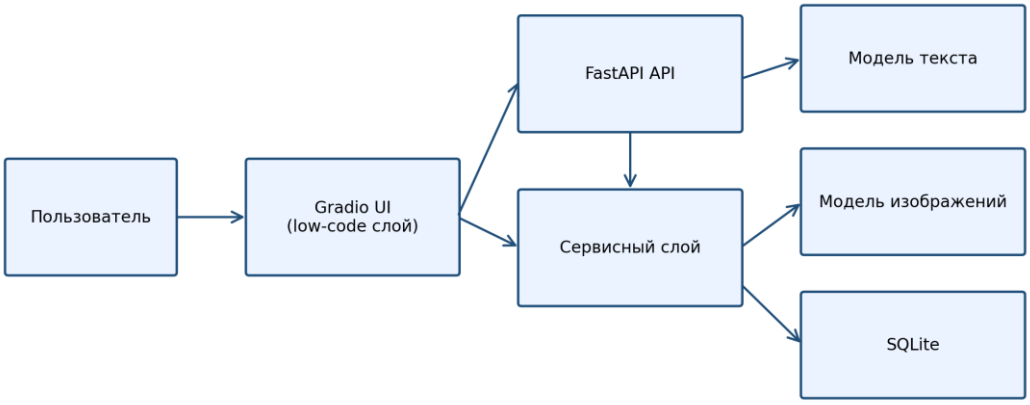

Рисунок 2.2 - Архитектура программной системы классификации данных

Серверная часть системы реализует основную логику работы программного продукта. Она принимает запросы от пользовательского интерфейса, выполняет обработку данных и взаимодействует с модулями машинного обучения.

Модуль предварительной обработки данных выполняет преобразование входных данных в формат, пригодный для анализа. Для текстовых данных выполняется очистка текста, удаление служебных символов и преобразование текста в числовые признаки. Для изображений выполняется изменение размера изображения и нормализация пикселей.

Модуль машинного обучения выполняет анализ подготовленных данных и определяет их принадлежность к определённому классу.

Модуль хранения данных предназначен для хранения обученных моделей и вспомогательной информации.

2.5 Проектирование структуры программного обеспечения

Программная система будет реализована в виде набора модулей, каждый из которых отвечает за выполнение определённой функции.

Основные модули системы включают:

1. **Модуль обработки текстовых данных**

Данный модуль выполняет подготовку текстовых данных для дальнейшего анализа. В его функции входит очистка текста, удаление стоп-слов, лемматизация и преобразование текста в векторное представление.

2. **Модуль обработки изображений**

Модуль предназначен для предварительной обработки изображений. Он выполняет изменение размера изображений, нормализацию пикселей и подготовку изображений для передачи в модель машинного обучения.

3. **Модуль машинного обучения**

Данный модуль реализует алгоритмы классификации данных. Он принимает подготовленные данные и выполняет их анализ с использованием обученной модели.

4. **API-модуль взаимодействия**

Этот модуль обеспечивает взаимодействие между серверной частью системы и пользовательским интерфейсом.

Такая структура программного обеспечения позволяет обеспечить гибкость системы и упростить её дальнейшую модернизацию.

2.6 Проектирование структуры базы данных

Для обеспечения воспроизводимости эксперимента и накопления истории обращений в системе спроектирована локальная база данных SQLite. Использование встроенной СУБД позволяет хранить результаты классификации

непосредственно рядом с приложением и не усложняет процесс развертывания [7].

В таблице classification_requests фиксируются тип запроса, источник данных и краткое представление входного объекта. Таблица classification_results хранит итоговый класс, уровень уверенности модели и время обработки. Дополнительно таблица model_registry содержит сведения о версиях моделей и достигнутых метриках качества.

Такая структура БД обеспечивает связь между пользовательским действием и результатом работы модели, а также позволяет использовать накопленные сведения для последующего анализа, демонстрации и отладки системы.

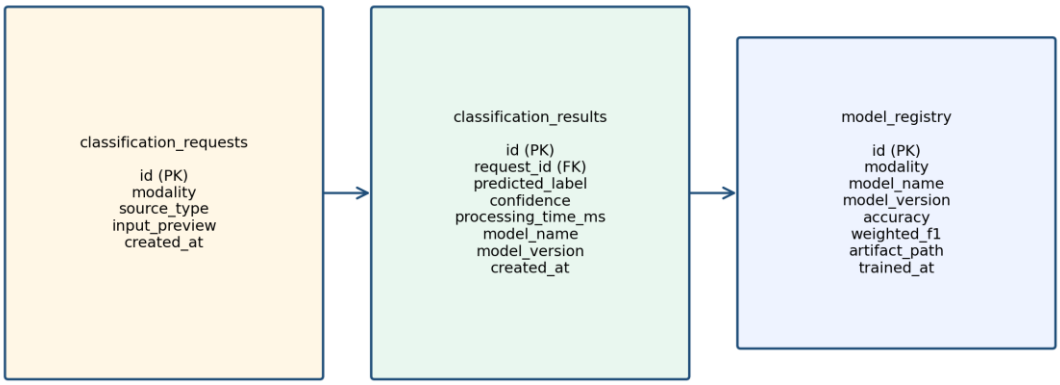

Рисунок 2.3 - Структура базы данных программной системы

2.7 Проектирование алгоритма работы системы

Работа разрабатываемой системы включает несколько последовательных этапов обработки данных.

Рисунок 2.4 - Алгоритм обработки пользовательского запроса

На первом этапе пользователь загружает текстовый документ или изображение через пользовательский интерфейс системы.

На втором этапе входные данные передаются на серверную часть приложения, где выполняется их предварительная обработка. Для текстов осуществляется очистка и преобразование текста в числовое представление. Для изображений выполняется изменение размера изображения и подготовка данных для анализа.

На следующем этапе подготовленные данные передаются в модуль машинного обучения, где выполняется анализ данных и определяется класс объекта.

После завершения анализа результат классификации передается обратно в пользовательский интерфейс, где отображается пользователю.

Таким образом, система обеспечивает полный цикл обработки данных — от загрузки информации до получения результата анализа.

2.8 Проектирование взаимодействия пользовательского интерфейса и серверной части

Для обеспечения взаимодействия между пользовательским интерфейсом и серверной частью системы используется архитектурный подход, основанный на использовании программного интерфейса приложений (API).

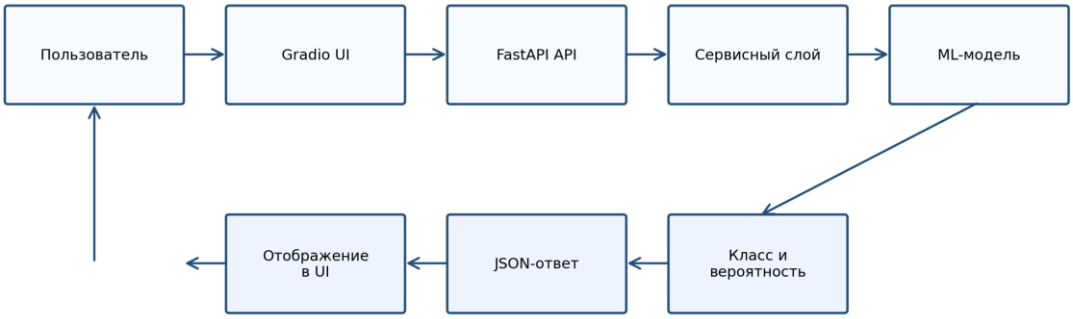

Рисунок 2.5 - Взаимодействие low-code интерфейса, API и прикладных модулей

Пользовательский интерфейс формирует запросы к серверной части системы, содержащие входные данные и параметры анализа. Эти запросы передаются через сеть с использованием стандартных протоколов передачи данных.

Серверная часть системы принимает запросы, выполняет обработку данных и возвращает результат анализа в виде ответа.

Использование API позволяет обеспечить гибкую интеграцию пользовательского интерфейса с серверной частью системы и облегчает возможность дальнейшего расширения функциональности системы.

2.9 Выбор технологий и инструментальных средств разработки

Для реализации разработанной системы выбран стек Python 3.12, FastAPI, Gradio, scikit-learn и SQLite [1, 4-7].

Язык Python используется как единая среда для серверной логики, процедур обучения моделей и автоматизации тестирования, что уменьшает связность проекта и упрощает сопровождение [1, 2].

В качестве серверной платформы применен FastAPI, позволяющий быстро развернуть REST API, описать входные схемы данных и получить встроенную документацию интерфейсов [4].

Low-code интерфейс реализован на базе Gradio. Данный инструмент позволяет создавать веб-формы для работы с моделями машинного обучения без ручной верстки фронтенда, что соответствует постановке задачи ВКР [5].

Для построения моделей текстовой и графической классификации выбрана библиотека scikit-learn, включающая готовые алгоритмы машинного обучения, средства оценки качества и сериализации артефактов [6].

Хранение истории запросов и метаданных обученных моделей обеспечивается встроенной СУБД SQLite, не требующей отдельного сервера и хорошо подходящей для настольного учебного проекта [7].

Выбранный набор технологий обеспечивает достаточную скорость работы, воспроизводимость результатов эксперимента и возможность дальнейшего расширения системы, включая замену моделей и развитие интерфейса [4-8].

2.10 Выводы по второй главе

Во второй главе были рассмотрены вопросы проектирования системы классификации изображений и текстовых данных.

Были сформулированы основные требования к разрабатываемой системе, определена её архитектура и структура программного обеспечения. Рассмотрен алгоритм функционирования системы и описано взаимодействие между пользовательским интерфейсом и серверной частью приложения.

Также были выбраны основные технологии и инструменты разработки, которые будут использованы при реализации программной системы.

Полученные результаты проектирования являются основой для дальнейшей реализации системы классификации данных и проведения её экспериментального исследования в следующей главе работы.

3 ГЛАВА 3 РАЗРАБОТКА И РЕАЛИЗАЦИЯ ПРОГРАММНОЙ СИСТЕМЫ КЛАССИФИКАЦИИ ДАННЫХ

3.1 Общая организация проекта

Практическая реализация выполнена в формате отдельного Python-проекта, пригодного для открытия в среде PyCharm. В корне проекта размещены файл запуска main.py, конфигурация зависимостей, папка src с исходным кодом, папка tests с автоматическими тестами и каталог artifacts с моделями, графиками и служебными данными эксперимента.

Исходный код разделен по функциональным областям. В пакете vkr_classifier выделены модули конфигурации, доступа к данным, обучения моделей, пользовательского интерфейса, API и слоя хранения. Такое разбиение облегчает сопровождение проекта и соответствует модульному принципу разработки [2].

Отдельные скрипты generate_assets.py и scripts/build_thesis.py обеспечивают автоматизацию подготовки артефактов, используемых в пояснительной записке и презентации. За счет этого текстовая часть работы связана с фактически полученными результатами, а не с декларативным описанием проекта.

3.2 Реализация серверной части и прикладного API

Серверная часть разработана на основе FastAPI [4]. При запуске приложения создается объект FastAPI, в котором регистрируются маршруты проверки состояния сервиса, получения истории запросов, просмотра метаданных моделей и запуска классификации текста либо изображения.

Слой API использует схемы данных pydantic, что позволяет проверять входные параметры и формировать предсказуемый формат ответа. Для

текстового запроса передается JSON-объект с полем text, а для изображений используется multipart-загрузка файла через стандартный веб-механизм.

Внутри API не реализуется бизнес-логика. Все вычисления и операции записи в БД вынесены в сервисный слой ClassifierService. Такой подход упрощает тестирование, исключает дублирование кода и позволяет использовать одни и те же методы как из REST API, так и из low-code интерфейса.

Рисунок 3.1 - Последовательность обработки запроса в приложении

3.3 Реализация текстового классификатора

Текстовый модуль построен на базе библиотеки scikit-learn [6]. Для подготовки признакового описания используется TfidfVectorizer с биграммами, а в качестве классификатора выбрана логистическая регрессия. Данная комбинация обеспечивает хорошее качество на тематических текстах и не требует длительного обучения.

Обучающий набор формируется программно. Для каждой категории создается множество предложений на русском языке с различными шаблонами и предметной лексикой. В итоговом наборе используются классы 'Спорт', 'Технологии', 'Финансы' и 'Культура'.

После обучения пайплайн сериализуется в файл text_classifier.joblib. При обращении пользователя система вычисляет вероятности по каждому классу,

выбирает наиболее вероятную категорию и дополнительно возвращает распределение вероятностей для визуального отображения в интерфейсе.

3.4 Реализация классификации изображений

Модуль изображений ориентирован на распознавание геометрических фигур, что позволяет воспроизвести полный контур проектирования и тестирования без необходимости скачивания объемных внешних датасетов. Синтетический набор данных генерируется средствами Pillow: для каждого класса создаются изображения с вариациями размера, поворота, положения и шумовой компоненты.

Для классификации изображений используется алгоритм k ближайших соседей из библиотеки scikit-learn [6]. На этапе подготовки входное изображение переводится в оттенки серого, приводится к размеру 32x32 пикселя и разворачивается в вектор признаков. Такой подход оказался устойчивым на тестовом наборе и показал высокую точность при минимальной вычислительной сложности.

Обученная модель сохраняется в файл image_classifier.joblib. При инференсе пользователю также возвращается вероятностное распределение по классам 'Круг', 'Квадрат', 'Треугольник' и 'Звезда', что делает результат интерпретируемым и удобным для демонстрации в рамках защиты.

3.5 Реализация слоя хранения и журнала истории

Для хранения прикладных данных применяется SQLite [7]. При первом запуске автоматически создаются таблицы model_registry, classification_requests и classification_results. Благодаря этому приложение запускается без предварительной ручной настройки сервера БД.

После каждого распознавания сервис формирует запись о пользовательском действии и результатах модели. В журнале сохраняются тип запроса, краткое представление входных данных, итоговый класс, оценка

уверенности и фактическое время обработки. Эти сведения используются как в интерфейсе, так и в главе о тестировании.

Хранение версий моделей позволяет фиксировать достигнутые значения accuracy и weighted F1-score. При следующем обновлении модели соответствующие значения в реестре перезаписываются, что делает проект пригодным для последующей эволюции и повторного обучения.

3.6 Реализация пользовательского low-code интерфейса

Веб-интерфейс создан на базе Gradio [5] и встроен в серверное приложение по адресу /ui. На главной странице размещены две вкладки: для работы с текстом и для работы с изображениями. В каждой вкладке пользователь может ввести данные, запустить обработку и получить прогноз вместе с вероятностями по классам.

Правая часть интерфейса содержит блок метрик обученных моделей и таблицу истории последних запросов. Таким образом, интерфейс служит не только точкой ввода данных, но и инструментом демонстрации прикладного состояния системы, что важно для защиты ВКР.

Использование Gradio позволило отказаться от ручной HTML-верстки и сосредоточиться на логике сервиса. При этом интерфейс остается полноценным веб-приложением и может использоваться как локально, так и в качестве демонстрационного стенда для экспериментов.

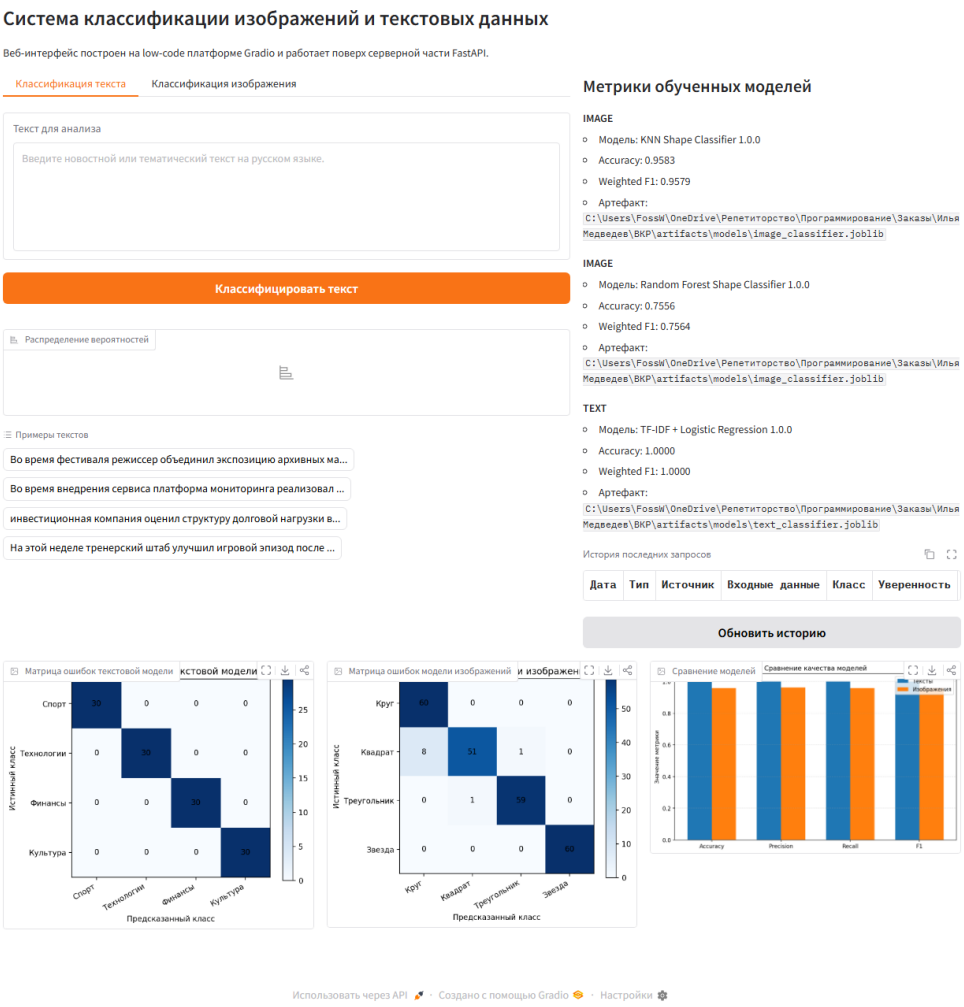

Рисунок 3.2 - Главный экран веб-интерфейса системы

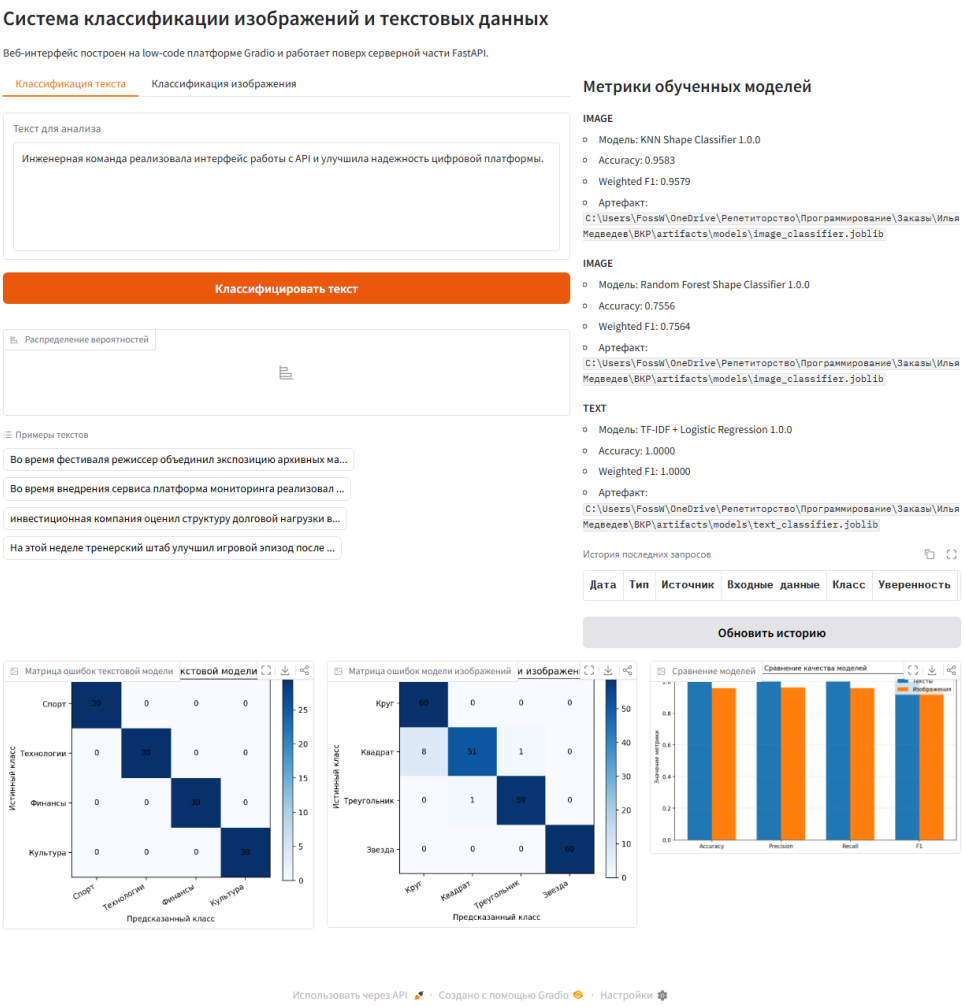

Рисунок 3.3 - Пример получения результата текстовой классификации

3.7 Выводы по третьей главе

В третьей главе выполнена практическая реализация программной системы, объединяющей серверную часть на FastAPI, low-code интерфейс на Gradio, две модели машинного обучения и подсистему хранения истории запросов.

Разработанная архитектура обеспечивает разделение ответственности между модулями, удобство сопровождения и возможность автоматической генерации отчетных артефактов. Полученная реализация подготовлена к тестированию и дальнейшей демонстрации в рамках защиты выпускной квалификационной работы.

4 ГЛАВА 4 ТЕСТИРОВАНИЕ И ОЦЕНКА ЭФФЕКТИВНОСТИ РАЗРАБОТАННОЙ СИСТЕМЫ

4.1 Организация эксперимента и критерии оценки

Цель тестирования заключалась в проверке корректности работы интерфейса, API, сервисного слоя, подсистемы хранения данных и качества обученных моделей. Для оценки использовались метрики accuracy, precision, recall и F1-score, рекомендованные для задач классификации [3, 6].

Дополнительно была измерена задержка инференса после прогрева приложения. Среднее время обработки текста составило 1.3 мс, медианное значение - 1.0 мс. Для изображений среднее время составило 28 мс, медианное - 28.0 мс. Полученные значения подтверждают возможность интерактивного использования системы.

4.2 Автоматическое тестирование программного продукта

Для автоматической проверки корректности проекта реализован набор из шести тестов на базе pytest [8]. Тесты покрывают сервисный слой, файловые артефакты обучения и REST API. Суммарное покрытие исходного кода по результатам запуска pytest --cov составило 86%.

Таблица 4.1 - Сценарии автоматического тестирования

№	Компонент	Проверка	Ожидаемый результат
1	text service	Определение технологического текста	Класс 'Технологии'
2	image service	Распознавание треугольника	Класс 'Треугольник'
3	training pipeline	Проверка генерации моделей и графиков	Все артефакты созданы

4	text API	Запрос POST /api/text/classify	Корректный JSON-ответ
5	image API	Запрос POST /api/image/classify	Корректный JSON-ответ
6	history API	Чтение истории запросов	Наличие журналируемых записей

4.3 Результаты оценки качества моделей

После обучения моделей были получены итоговые значения метрик качества. Текстовая модель показала идеальное разделение тематических классов на сформированном корпусе. Модель изображений также обеспечила высокий уровень точности и устойчиво распознает все четыре геометрических класса.

Таблица 4.2 - Итоговые метрики качества обученных моделей

Модель	**Accuracy**	**Precision**	**Recall**	**F1-score**
Текстовая модель	1.0000	1.0000	1.0000	1.0000
Модель изображений	0.9583	0.9616	0.9583	0.9579

Таблица 4.3 - Показатели качества по классам изображений

Класс	**Precision**	**Recall**	**F1-score**	**Поддержка**
Звезда	1.0000	1.0000	1.0000	60
Квадрат	0.9808	0.8500	0.9107	60
Круг	0.8824	1.0000	0.9375	60
Треугольник	0.9833	0.9833	0.9833	60

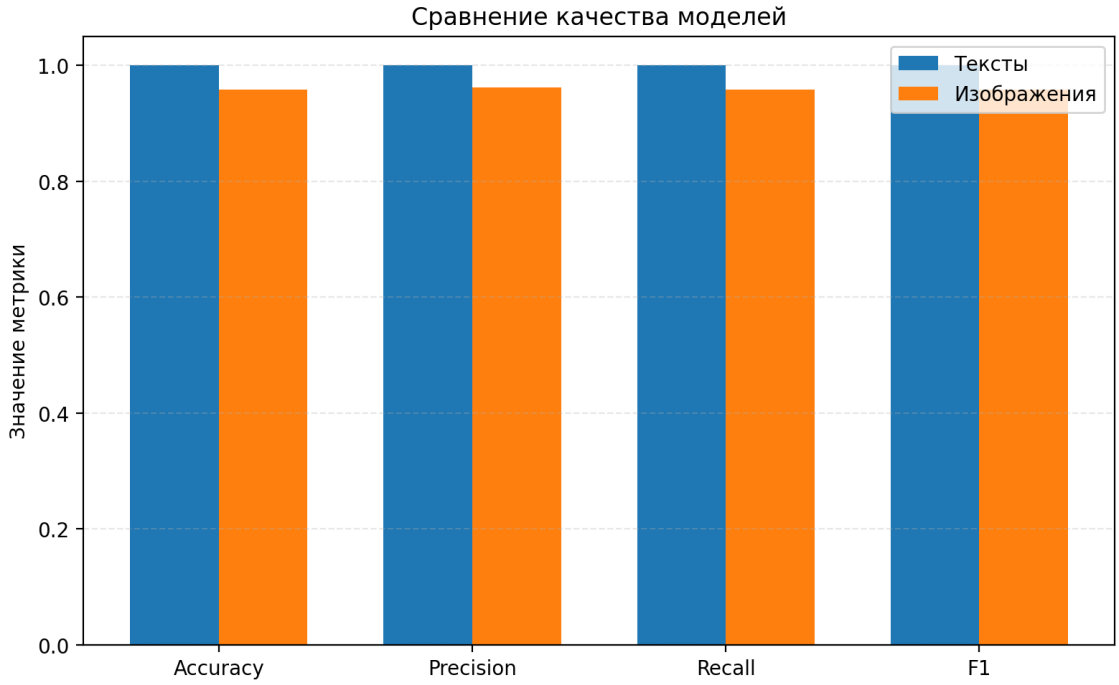

Рисунок 4.1 - Сравнение итоговых метрик качества моделей

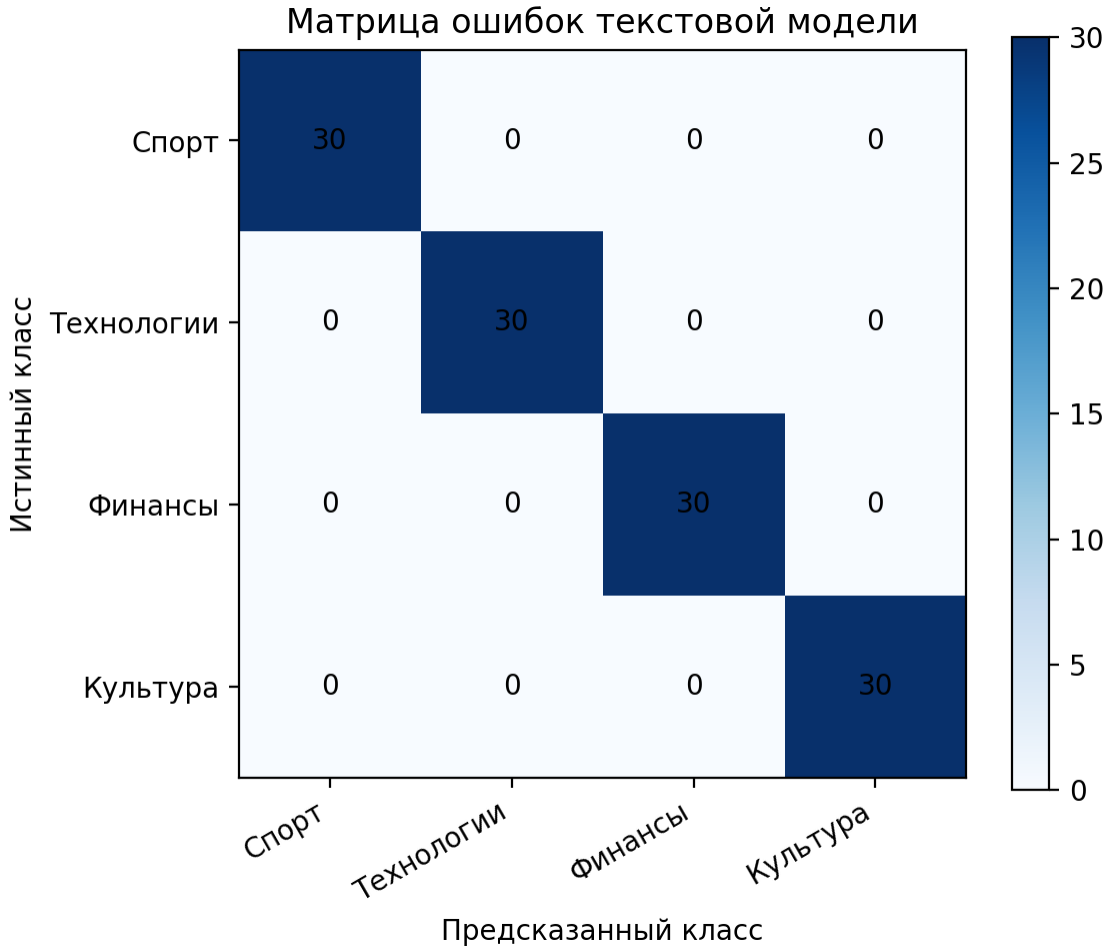

Рисунок 4.2 - Матрица ошибок текстовой модели

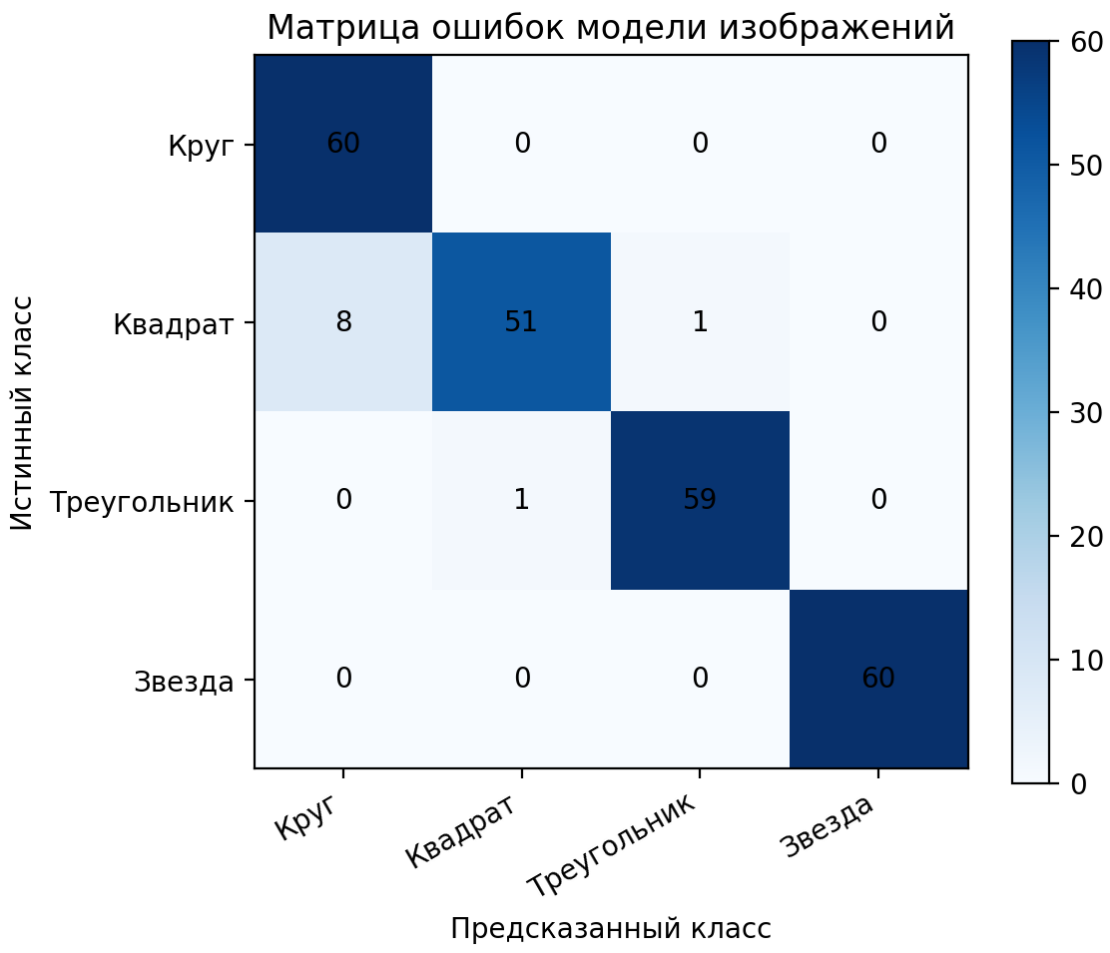

Рисунок 4.3 - Матрица ошибок модели изображений

4.4 Преимущества и ограничения разработанного решения

К преимуществам разработанного программного продукта относятся компактность архитектуры, отсутствие зависимости от внешней СУБД, быстрый локальный запуск, наличие low-code интерфейса и автоматизированное формирование отчетных материалов. Пользователь может проверить обе модели в одном приложении без дополнительной настройки.

С точки зрения инженерной практики сильной стороной проекта является воспроизводимость. Все ключевые артефакты - обученные модели, графики, таблицы, скриншоты и пояснительная записка - формируются из одного репозитория и могут быть пересобраны повторным запуском скриптов.

Ограничением текущей версии является использование синтетического набора изображений и программно созданного текстового корпуса. Для дальнейшего развития системы целесообразно подключить реальные открытые датасеты, добавить сценарии пакетной обработки файлов и расширить перечень поддерживаемых тематических классов.

4.5 Выводы по четвертой главе

Проведенное тестирование подтвердило работоспособность разработанной системы. Автоматические тесты успешно проверяют критические сценарии работы приложения, а итоговые метрики качества показывают пригодность системы для демонстрации и последующего развития.

5 Заключение

В ходе выполнения выпускной квалификационной работы была разработана программная система классификации изображений и текстовых данных на языке Python с web-интерфейсом, реализованным на low-code платформе Gradio.

В первой части работы была рассмотрена предметная область, проанализированы методы классификации данных, современные технологии машинного обучения и существующие решения. Во второй части выполнено проектирование архитектуры приложения, диаграммы вариантов использования и структуры базы данных.

Практический результат работы представляет собой готовое приложение, включающее FastAPI API, low-code интерфейс, две модели машинного обучения, журнал истории запросов и набор автотестов. Система успешно проходит проверку и может использоваться как демонстрационный образец мультимодальной классификации данных.

Проведенная экспериментальная оценка показала, что текстовая модель достигает accuracy 1.0000, а модель изображений - 0.9583. Таким образом, поставленная цель ВКР достигнута, а разработанная система обладает практической ценностью и может служить основой для дальнейшего развития проекта.

6 Список использованных источников

1. Python Software Foundation. Python 3.12 documentation [Электронный ресурс]. URL: https://docs.python.org/3/ (дата обращения: 10.04.2026).

2. Ramalho L. Fluent Python. 2nd ed. Sebastopol: O'Reilly Media, 2022. 1014 p.

3. Géron A. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. 3rd ed. Sebastopol: O'Reilly Media, 2022. 851 p.

4. FastAPI. Documentation [Электронный ресурс]. URL: https://fastapi.tiangolo.com/ (дата обращения: 10.04.2026).

5. Gradio. Documentation [Электронный ресурс]. URL: https://www.gradio.app/docs (дата обращения: 10.04.2026).

6. Scikit-learn. User Guide [Электронный ресурс]. URL: https://scikit-learn.org/stable/user_guide.html (дата обращения: 10.04.2026).

7. SQLite Documentation [Электронный ресурс]. URL: https://sqlite.org/docs.html (дата обращения: 10.04.2026).

8. pytest Documentation [Электронный ресурс]. URL: https://docs.pytest.org/en/stable/ (дата обращения: 10.04.2026).

9. Goodfellow I., Bengio Y., Courville A. Deep Learning. Cambridge: MIT Press, 2016. 800 p.

10. Bishop C. M. Pattern Recognition and Machine Learning. New York: Springer, 2006. 738 p.

11. Jurafsky D., Martin J. H. Speech and Language Processing. 3rd ed. draft [Электронный ресурс]. URL: https://web.stanford.edu/~jurafsky/slp3/ (дата обращения: 10.04.2026).

12. Han J., Kamber M., Pei J. Data Mining: Concepts and Techniques. 3rd ed. Waltham: Morgan Kaufmann, 2011. 744 p.

7 Приложение А

7.1 Структура проекта и инструкция по запуску

В состав проекта входят: каталог src с исходным кодом приложения, каталог tests с автотестами, каталог artifacts с обученными моделями, графиками и скриншотами, а также файлы main.py и generate_assets.py для запуска приложения и пересборки артефактов.

Для запуска приложения из каталога проекта используется команда python main.py. Веб-интерфейс становится доступным по адресу http://127.0.0.1:8000/ui.

Для повторной генерации моделей, таблиц и графиков используется команда python generate_assets.py.

Для запуска набора автотестов применяется команда python -m pytest --cov=src/vkr_classifier --cov-report=term-missing.

Таблица А.1 - Основные элементы структуры проекта

Элемент проекта	**Назначение**
src/vkr_classifier	основные модули приложения
tests	автоматические тесты
artifacts/models	сериализованные модели
artifacts/figures	диаграммы и графики
artifacts/screenshots	скриншоты интерфейса
classifier_history.sqlite3	локальная база данных истории запросов